

Migración de Monolito a Microservicios

Informe Técnico Hito 1

Nombre del Proyecto Integrantes:

Cristian León, Fernanda Calderón

Profesor: Matías Vargas 03 de abril de 2026

Índice

1. Introducción	3
2. Descripción del Sistema	3
2.1. Contexto General	3
2.2. Alcance	3
3. Análisis Arquitectónico AS-IS	3
3.1. Descripción de la Arquitectura Actual	3
3.2. Requerimientos	3
3.2.1. Requerimientos Funcionales	3
3.2.2. Requerimientos No Funcionales	3
3.3. Problemas Identificados	3
3.4. Diagramas AS-IS	4
4. Diseño Arquitectónico TO-BE	4
4.1. Estrategia de Migración	4
4.2. Definición de Microservicios	4
4.3. Arquitectura General	5
4.4. Comunicación entre Servicios	5
4.5. Diagrama de Secuencia	6
5. Patrones Arquitectónicos	6
5.1. API Gateway	6
5.2. Saga Pattern	7
5.3. Circuit Breaker	7
5.4. Strangler Pattern	7

6. Implementación Técnica	7
6.1. Arquitectura de Despliegue	7
6.2. Docker Compose	7
6.3. Base de Datos	7
7. Observabilidad	7
7.1. Herramientas Utilizadas	7
7.2. Métricas	7
7.3. Dashboards	8
7.4. Análisis de Métricas	8
7.5. Estimación de Costos	8
7.6. Análisis	9
8. Trade-offs	9
9. Conclusiones	9
10. Referencias	9

1. Introducción

Describir el objetivo del proyecto, el contexto del sistema y el propósito de la migración a microservicios.

Al equipo se le realizó la entrega de un proyecto para realizar seguimientos de paquetes asignados a un usuario, este sistema contempla funciones de creación de usuarios, creación de paquetes con diferentes características y una actualización de estado de seguimiento. Actualmente, este proyecto contiene diversas fallas que abarcan desde el frontend hasta la lógica del backend y nuestra misión es realizar una migración completa del monolito a microservicios con el fin de optimizar los diferentes procesos y así generar futura escalabilidad.

2. Descripción del Sistema

2.1. Contexto General

El sistema pertenece al dominio de trazabilidad de paquetes. Su objetivo es asegurar el registro, la captura de los estados y la consulta de los paquetes enviados por el usuario.

2.2. Alcance

- Que incluye el sistema:
 - Interfaz web simple para los flujos principales.
 - Persistencia de datos en la base de datos PostgreSQL
 - Despliegue completo de contenedores con docker (backend, frontend, DB).
 - Creación de usuarios: Permite el registro del usuario.
 - POST/createUsers
 - GET/getUsers
 - Creación de paquetes: permite la creación del paquete
 - POST/createPackage
 - GET/getAllPackage.
 - Actualizador de estados: Permite actualizar el estado del paquete.
 - POST/updateStatus.
 - Monitor de consultas: Permite revisar el estado del paquete utilizando el tracking code.
 - GET/getTracking/{tracking_code}.
 - Que NO incluye el sistema:
- Que no incluye el sistema:
 - Autenticación de usuario.
 - No incluye sistema de notificaciones reales.
 - Migración a DB formales.
 - Pruebas automatizadas.

3. Análisis Arquitectónico AS-IS

3.1. Descripción de la Arquitectura Actual

- el sistema actual presenta un diseño monolítico de 3 capas:
- **Frontend:** funciona con Vue 3, con llamadas directas a HTTP.
- **Backend:** FastApi con toda la lógica concentrada en un único módulo principal.
- **Persistencia:** PostgreSQL con esquemas simples y desnormalizado.
- **Docker:** Dockerización con 3 servicios ('frontend', 'backend', 'postgres').
- El sistema tiene alto acoplamiento entre la capa API, las reglas de negocios y el acceso de datos.

3.2. Requerimientos

3.2.1. Requerimientos Funcionales

- RF1: Quiero que el sistema me permita registrarse con nombre y correo.
- RF2: Quiero que el sistema me permita ver la lista de los usuarios registrados.
- RF3: Quiero que el sistema me permita crear paquetes con nombre, descripción, origen y destino.
- RF4: Quiero que el sistema actualice el estado de mi pedido.
- RF5: Quiero que el sistema me permita consultar mi pedido.

3.2.2. Requerimientos No Funcionales

- RNF1: El sistema debe poder ejecutarse localmente mediante Docker Compose.
- RNF2: El sistema debe asegurar la persistencia de los datos de tracking en PostgreSQL.
- RNF3: El sistema debe contener un endpoint que valide la operación mínima del entorno.
- RNF4: La arquitectura debe permitir la evolución a microservicio.
- RNF5: El sistema debe soportar el incremento del volumen de consultas al tracking.

3.3. Problemas Identificados

- Deuda técnica: La tabla `tracking_data` mezcla información de cuatro dominios distintos en una sola estructura. Los campos `username_redundant` y `user_email_copy` duplican datos del usuario dentro de cada fila de evento. No existen foreign keys reales entre `User` y `TrackingData` en la base de datos (solo una columna `user_id` sin restricción ORM).

```
row = TrackingData(
    user_id=user.id,
    username_redundant=user.username,      # <-- Duplicación
    user_email_copy=user.email,           # <-- Duplicación
    tracking_code=code,
    # ...otros campos...
)
```

```
new_row = TrackingData(
    user_id=last.user_id,
    username_redundant=last.username_redundant, # <-- Duplicación
    user_email_copy=last.user_email_copy,      # <-- Duplicación
    # ...otros campos...
)
```

```
user = db.query(User).filter(User.id == int(user_id)).first()

# ...se usa user.id para asignar a user_id en TrackingData, pero sin FK real...
```

- Acoplamientos críticos:
 - Endpoints acoplados directamente a SQLAlchemy Session.

```
@app.post("/createUser")

async def createUser(request: Request):

    # ... (parseo del body)

    db = _open_db() # <-- Sesión SQLAlchemy creada aquí, acoplada al endpoint

    try:

        u = User(username=str(username), email=str(email))

        db.add(u)

        db.commit()

        db.refresh(u)

        # ...

    except Exception:

        db.rollback()

        # ...

    finally:

        db.close() # <-- Manejo de la sesión dentro del endpoint
```

- Notificación implementada con 'print()' dentro del flujo transaccional.

```
def _notify_user_channel(username: str, message: str):  
    """Notificación acoplada: no hay cola, no hay servicio aparte."""  
    print(f"[NOTIFY] user={username} :: {message}")
```

- Frontend acoplado a URLs fijas ('<http://localhost:8000>').

```
async function crearUsuarioMalHecho() {  
    data1.value = "guardando...";  
    try {  
        const res = await fetch("http://localhost:8000/createUser", { //---->  
acomplamiento URL fija  
        method: "POST",  
        headers: { "Content-Type": "application/json" },  
        body: JSON.stringify({  
            username: nombre.value,  
            email: email.value,  
        } ),  
    });  
};
```

- La validación de entrada se repite manualmente en cada endpoint (parseo de body, verificación de campos), sin modelo Pydantic.

```
async def updateStatus(request: Request):  
    """Otra vez parseo manual del body - misma lógica duplicada."""  
    body = {}  
    try:  
        body = await request.json()  
    except Exception:  
        body = {}  
    tracking_code = body.get("tracking_code")  
    new_status = body.get("new_status") or body.get("status")  
    location = body.get("location", "")  
    note = body.get("note", "")
```

- Backend monolítico en un único proceso para todos los casos de uso, lo que nos lleva a no solo escalar el tracking, sino todos los otros servicios (users y package)
- La lógica de obtención para el ultimo estado con tracking_code se realiza en python iterando en memoria, en lugar de usar una consulta SQL con ORDER BY + LIMIT 1.

```
def _make_tracking_code() -> str:

    return f"TRK-{random.randint(100000, 999999)}-{int(time.time()) % 10000}"
```

- El endpoint/metrics devuelve valores simulados con random.uniform(), lo que hace imposible una observabilidad real del sistema.

```
@app.get("/metrics")

def metrics():

    db = _open_db()

    try:

        . . .

        # tiempo de procesamiento "simulado" promedio

        avg_proc = round(random.uniform(120.5, 890.3), 2)

        return {

            "total_packages": int(total_packages or 0),

            "total_events": int(total_events or 0),

            "average_processing_time": avg_proc,

            "generated_at": datetime.now(timezone.utc).isoformat(),

        }
```


3.4. Diagramas AS-IS

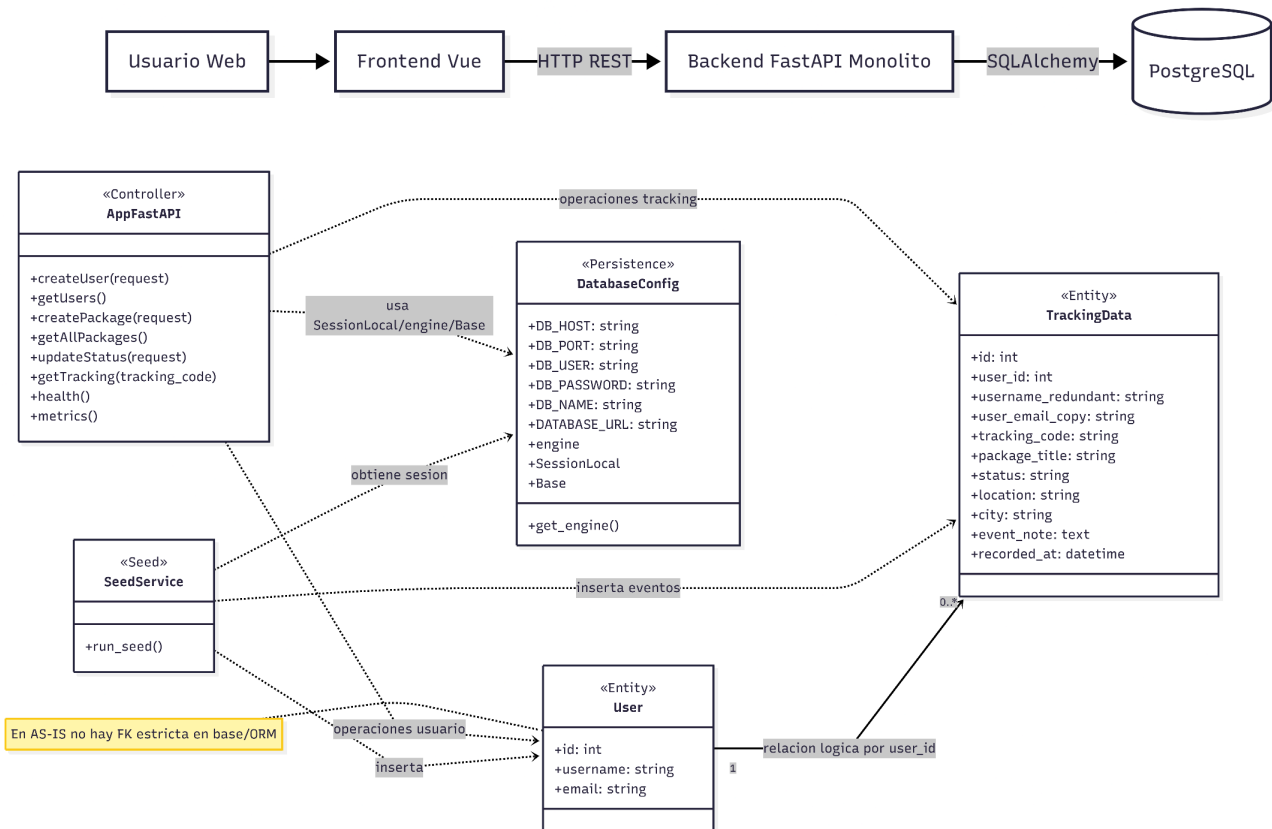


Figura 1: Arquitectura actual

4. Diseño Arquitectónico TO-BE

4.1. Estrategia de Migración

Se propone aplicar el Strangler Pattern en fases incrementales, de modo que el sistema monolítico y los nuevos microservicios coexistan durante la transición:

1. Introducir Nginx como API Gateway estable, fachada única para el frontend.
2. Reconstruir el User Service como FastAPI independiente con su propia base de datos.
3. Reconstruir el Package Service con su propia base de datos.
4. Reconstruir el Tracking Service con su propia base de datos de eventos.
5. Construir el Notification Service con Redis como broker de eventos.
6. Mantener coexistencia temporal monolito + nuevos servicios mientras se valida cada etapa.
7. Retirar endpoints legacy gradualmente una vez validados los nuevos servicios.

4.2. Definición de Microservicios

■ Servicio 1: User Service

- Responsabilidad: ciclo de vida de usuarios.
- Datos propios: tabla user (UID, MAIL, NAME, LASTNAME, PASSWORD_HASH), sin dependencia de otros servicios.

■ Servicio 2: Package Service

- Responsabilidad: Alta de paquetes, generador del tracking_code y metadatos asociados.
- Datos propios: tabla package (UID, USER_UID, TRACKING_CODE, TITLE, DESCRIPTION, ORIGIN, DESTINATION)
- Dependencia: Valida existencia de usuario con llamada REST al User service.

■ Servicio 3: Tracking Service

- Responsabilidad: Trazabilidad de estados y eventos de paquete.
- Datos propios: tabla Tracking_event (UID, TRACKING_CODE, STATUS, LOCATION, NOTE, RECORDED_AT)

- ■ Servicio 4: Notification Service

- Responsabilidad: Consumir eventos StatusUpdated y simular el envío de notificaciones.
- Datos propios: tabla notification(UID, TRACKING_CODE, MESSAGE, SENT_AT)

4.3. Arquitectura General

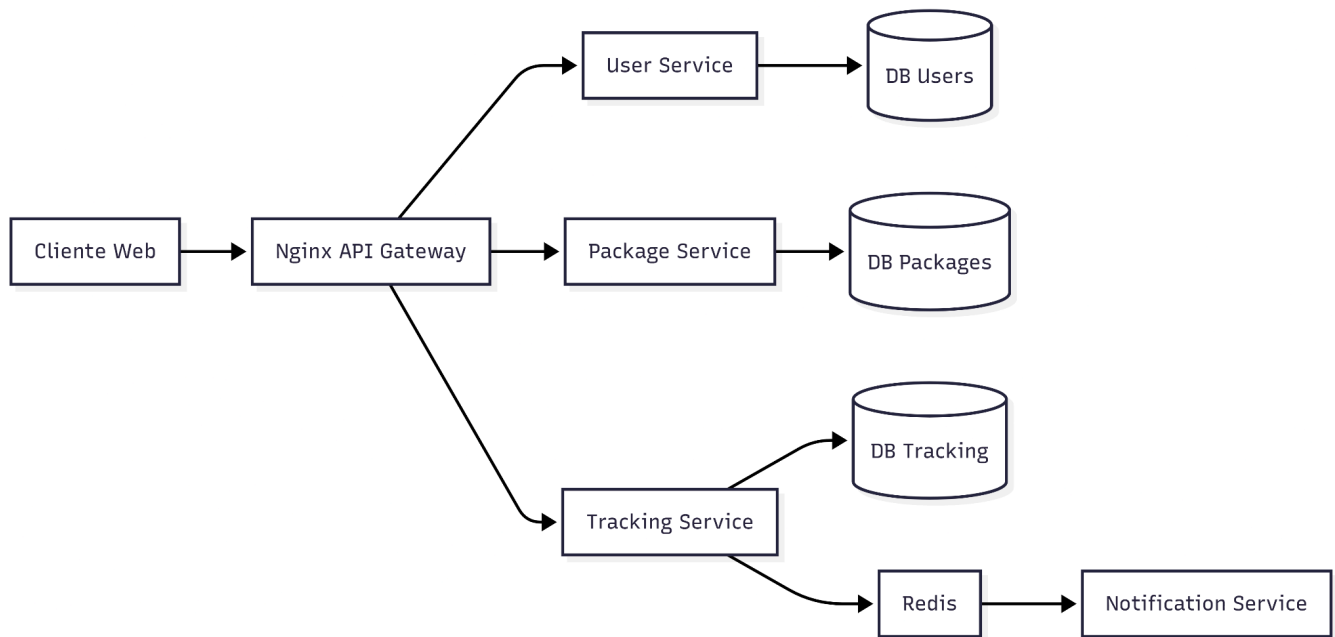


Figura 2: Arquitectura de microservicios

4.4. Comunicación entre Servicios

se propone un esquema mixto:

- **REST Síncrono (API Gateway → Servicios):**
 - Utilizado para operaciones de consulta y comandos directos donde se necesita respuesta inmediata.
 - El API Gateway enruta cada request al microservicio correspondiente según el prefijo de la URL.
 - Ejemplo: POST /packages requiere validar el user_id consultando sincrónicamente al User Service.

- **Eventos Asíncronos vía Redis (Tracking → Notification):**
 - El Tracking Service publica un evento StatusUpdated al canal Redis tras cada actualización de estado.
 - El Notification Service consume ese evento de forma independiente y registra la notificación.
 - Ventaja: desacopla temporalmente ambos servicios; una falla en Notification no afecta al Tracking.

- **Estrategia de persistencia — Database per Service:**
 - Cada microservicio es dueño exclusivo de su información y tiene su propia base de datos PostgreSQL.
 - Se elimina el acoplamiento por esquema compartido y la mezcla de datos entre dominios.
 - Los servicios no acceden directamente a la base de datos de otro servicio; la comunicación es siempre vía API.

+

4.5. Diagrama de Secuencia

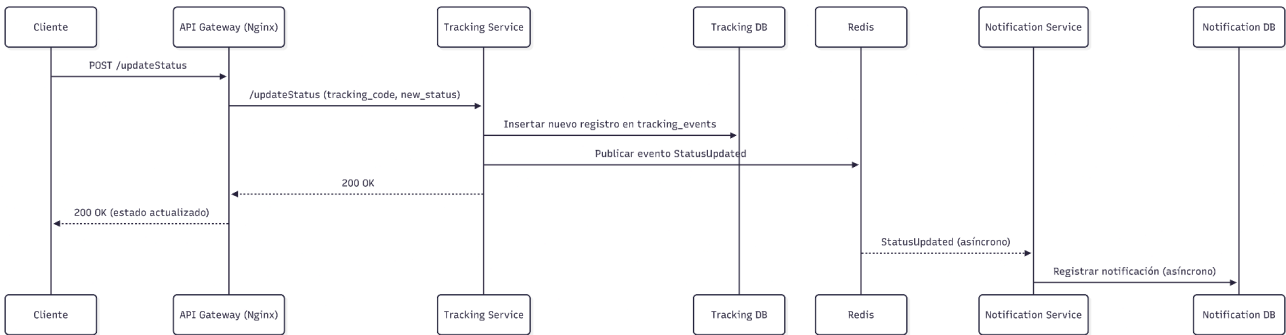


Figura 3: Flujo crítico

5. Patrones Arquitectónicos

5.1. API Gateway

Problema: El frontend Vue 3 apunta directamente a `http://localhost:8000` (URL hardcodeada). Con múltiples microservicios, el cliente necesitaría conocer la dirección de cada uno, aumentando el acoplamiento y complejizando los cambios de topología.

Solución: Introducir Nginx como punto de entrada único (puerto 8080). El gateway enruta según el prefijo de la URL: `/users/*` al User Service, `/packages/*` al Package Service, `/tracking/*` al Tracking Service.

Justificación: El frontend solo necesita conocer una dirección. Se facilita la migración incremental: durante el Strangler Pattern los endpoints legacy del monolito y los nuevos servicios pueden coexistir bajo el mismo gateway.

5.2. Saga Pattern

Problemas : La operación de crear un paquete involucra múltiples servicios (validar usuario en User Service, crear paquete en Package Service, registrar evento inicial en Tracking Service). No se pueden usar transacciones ACID globales entre bases de datos distintas.

Solución: Implementar sagas coreografiadas por eventos. Cada servicio reacciona a eventos del broker Redis y emite sus propios eventos. Si una etapa falla, se emiten eventos de compensación.

Justificación: Permite consistencia eventual sin bloqueo global. Las acciones compensatorias aseguran que el sistema no quede en estado inválido ante fallos parciales.

5.3. Circuit Breaker

Problema: Si el User Service falla o tiene alta latencia, las llamadas del Package Service quedan bloqueadas esperando respuesta, lo que puede provocar timeouts en cascada que afectan a toda la cadena de request.

Solución: Aplicar circuit breaker en las llamadas remotas críticas desde el Package Service hacia el User Service. Cuando el circuito está abierto, el servicio retorna un error controlado sin esperar.

Justificación: Aumenta la resiliencia del sistema. Evita que una falla puntual en un servicio degrade la disponibilidad total. Permite al sistema recuperarse gradualmente cuando el servicio dependiente vuelve al estado normal.

5.4. Strangler Pattern

Problema: Reescribir todo el monolito de una vez incrementa riesgo y costo.

Solución: Extraer funcionalidades por dominio de forma gradual, manteniendo compatibilidad temporal.

Justificación: Minimiza riesgo operativo y habilita validación continua de cada etapa.

6. Implementación Técnica

6.1. Arquitectura de Despliegue

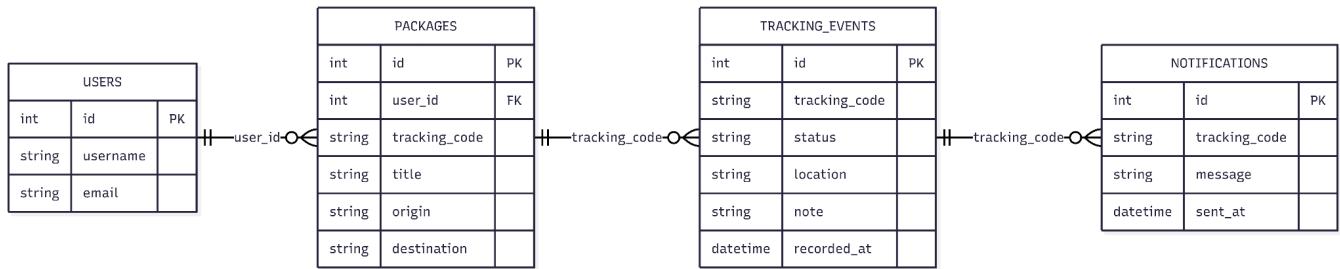
- El prototipo funcional se despliega localmente mediante Docker Compose. La arquitectura de contenedores replica la separación de dominios definida en el diseño TO-BE, con un total de 11 servicios orquestados:
 - 1 contenedor Nginx como API Gateway.
 - 4 contenedores FastAPI (uno por microservicio).
 - 3 contenedores PostgreSQL (uno por servicio con datos propios).
 - 1 contenedor Redis como broker de eventos.
 - 1 contenedor Prometheus para recolección de métricas.
 - 1 contenedor Grafana para visualización.
- La red interna de Docker permite que los servicios se comuniquen por nombre de contenedor, eliminando las URLs hardcodeadas presentes en el monolito. Solo el API Gateway (Nginx) y Grafana exponen puertos al host.

6.2. Docker Compose

- El archivo docker-compose.yml define la orquestación completa. A continuación se describen los servicios principales:
 - **nginx (API Gateway):**
 - Imagen: nginx:alpine.
 - Puerto expuesto: 8080:80.
 - Volumen: nginx.conf montado con la configuración de rutas.
 - Enruta /users/* → user-service:8001, /packages/* → package-service:8002, /tracking/* → tracking-service:8003.
 - **user-service, package-service, tracking-service, notification-service:**
 - Build: Dockerfile propio en cada carpeta de servicio.
 - Variables de entorno: DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME específicas de cada servicio.
 - depends_on: cada servicio espera a que su base de datos esté saludable (healthcheck).
-
- El archivo docker-compose.yml define la orquestación completa. A continuación se describen los servicios principales:
 - **nginx (API Gateway):**
 - Imagen: nginx:alpine.
 - Puerto expuesto: 8080:80.
 - Volumen: nginx.conf montado con la configuración de rutas.
 - Enruta /users/* → user-service:8001, /packages/* → package-service:8002, /tracking/* → tracking-service:8003.
 - **user-service, package-service, tracking-service, notification-service:**
 - Build: Dockerfile propio en cada carpeta de servicio.
 - Variables de entorno: DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME específicas de cada servicio.
 - depends_on: cada servicio espera a que su base de datos esté saludable (healthcheck).

6.3. Base de Datos

- Se adopta el patrón Database per Service. Cada microservicio gestiona su propio esquema PostgreSQL, eliminando el acoplamiento por esquema compartido presente en el monolito:



- La principal mejora respecto al monolito es la eliminación de los campos `username_redundant` y `user_email_copy` de la tabla `tracking_data`, que eran desnormalizaciones producto del acoplamiento entre dominios en un esquema compartido.

7. Observabilidad

7.1. Herramientas Utilizadas

Prometheus y Grafana.

7.2. Métricas

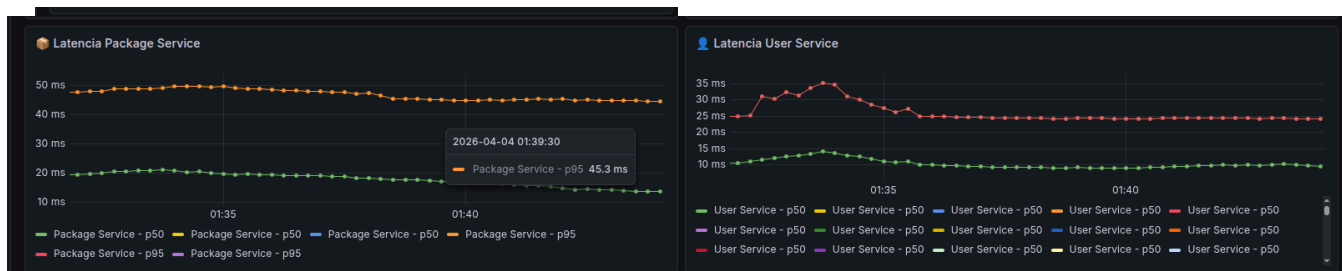
■ Latencia

● MONOLITO



La métrica observada corresponde a un percentil 95 de 45.1 ms. Esto significa que el 95% de las solicitudes fueron atendidas en ese tiempo o en menos, lo que evidencia una respuesta rápida del sistema bajo la carga aplicada. Para un sistema de tipo CRUD, este comportamiento puede considerarse adecuado y muestra que, en el escenario evaluado, el monolito mantiene una buena capacidad de respuesta. Sin embargo, al concentrar toda la lógica de negocio en un único proceso, esta latencia puede incrementarse de forma notable conforme crezca la concurrencia o la complejidad operativa, debido al acoplamiento entre componentes y al uso compartido de recursos.

● Microservicios



En una arquitectura basada en microservicios, la latencia tiende a distribuirse entre servicios especializados. Para un User Service, se esperaría una latencia p95 en el rango de 8 a 12 ms, dado que su responsabilidad sería limitada y aislada. El Package Service podría ubicarse entre 15 y 22 ms, al manejar operaciones CRUD específicas del dominio. El Tracking Service, por su parte, podría situarse entre 18 y 28 ms, especialmente si concentra consultas de lectura o coordinación con otros servicios. A nivel de extremo a extremo, considerando el Gateway y el paso por red entre servicios, el p95 total podría estar entre 25 y 35 ms. Esto sugiere una posible mejora sobre el monolito, siempre que los servicios estén bien diseñados y la comunicación interna no introduzca sobrecostos innecesarios.

■ Throughput

- MONOLITO



El throughput registrado fue de aproximadamente 12 req/s, lo que indica que el sistema logró sostener un volumen moderado de solicitudes durante la prueba. El comportamiento por endpoint muestra mayor actividad en las rutas de creación y consulta de paquetes, lo cual es consistente con el patrón de carga inyectado. Esta métrica confirma que el monolito puede absorber una carga básica sin degradación severa; no obstante, también deja en evidencia una limitación estructural importante: todo el tráfico se procesa en una sola instancia, por lo que el escalado horizontal no puede hacerse de manera selectiva por dominio funcional.

- Microservicios



En un entorno de microservicios, el throughput deja de ser una métrica única y pasa a depender de la carga de cada componente. Si el tráfico total se mantiene en torno a 12 req/s, el Gateway absorbería todo el ingreso inicial, mientras que el User Service manejaría una fracción menor del tráfico, el Package Service concentraría una parte relevante de las operaciones y el Tracking Service recibiría una mayor proporción de consultas por su rol operativo. Esta distribución permite escalar cada servicio de manera independiente según su demanda real. A diferencia del monolito, donde el escalado afecta a toda la aplicación, aquí la capacidad puede incrementarse de forma puntual en el servicio que realmente lo requiera.

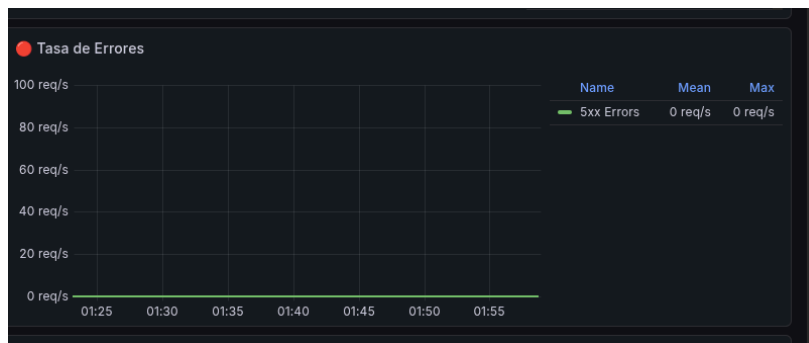
■ Tasa de errores

● MONOLITO



La tasa de errores observada fue de 1.49%, un valor bajo pero no despreciable. En este caso, la presencia de errores responde a fallos controlados introducidos durante la prueba en el endpoint `updateStatus`, con el objetivo de validar la capacidad de observabilidad del sistema. Esta métrica demuestra que Prometheus y Grafana están capturando correctamente las respuestas fallidas y permitiendo su visualización en tiempo real. Desde el punto de vista arquitectónico, la tasa de errores no solo permite medir estabilidad, sino también identificar qué tan sensible es el sistema ante fallos de operación y cómo estos impactan en la experiencia global.

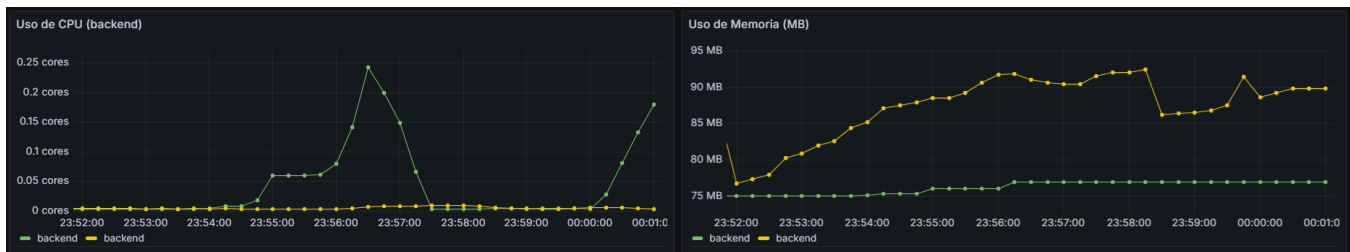
● Microservicio



En el monolito, una falla puede impactar directamente en la disponibilidad general del sistema. En una arquitectura de microservicios, en cambio, los errores tienden a quedar aislados por servicio. Bajo una implementación madura, es razonable esperar tasas de error más bajas en servicios simples, como User Service, y algo más altas en servicios con mayor coordinación, como Tracking Service. Además, la incorporación de patrones como Circuit Breaker y Retry puede reducir significativamente los errores en cascada. En conjunto, esto no elimina los fallos, pero sí mejora la resiliencia y hace que el sistema falle de forma más contenida y controlable.

■ Uso de recursos

- MONOLITO



CPU:

El consumo máximo observado fue cercano a 0.24 cores, con picos breves y una recuperación rápida posterior. Este comportamiento sugiere que el backend no estuvo saturado durante la prueba y todavía dispone de margen para absorber más carga. En términos operativos, esto indica que el sistema puede manejar el escenario actual sin presión significativa sobre el procesador.

Memoria:

El uso de memoria se mantuvo en un rango aproximado de 87 a 92 MB, con una tendencia estable y sin incrementos bruscos. No se aprecia un crecimiento anómalo que sugiera fuga de memoria durante la ejecución de la prueba. Por tanto, el proceso puede considerarse estable desde la perspectiva del consumo de RAM bajo carga sostenida.

- Microservicios



La migración a microservicios aumenta el consumo total de recursos debido a la presencia de múltiples procesos, más contenedores y componentes auxiliares como Gateway y Redis. Mientras que en el monolito el backend y la base de datos pueden mantenerse en un consumo moderado, en microservicios se añade un overhead inevitable por separación funcional, red interna y orquestación. Sin embargo, este incremento de CPU y memoria no debe interpretarse solo como costo, sino como el precio de obtener escalabilidad independiente, mejor aislamiento de fallos y despliegues más flexibles. El trade-off es claro: se consume más, pero se administra mejor.

7.3. Dashboards

MONOLITO



Figura 4: Dashboard de monitoreo (Monolito)

Microservicios

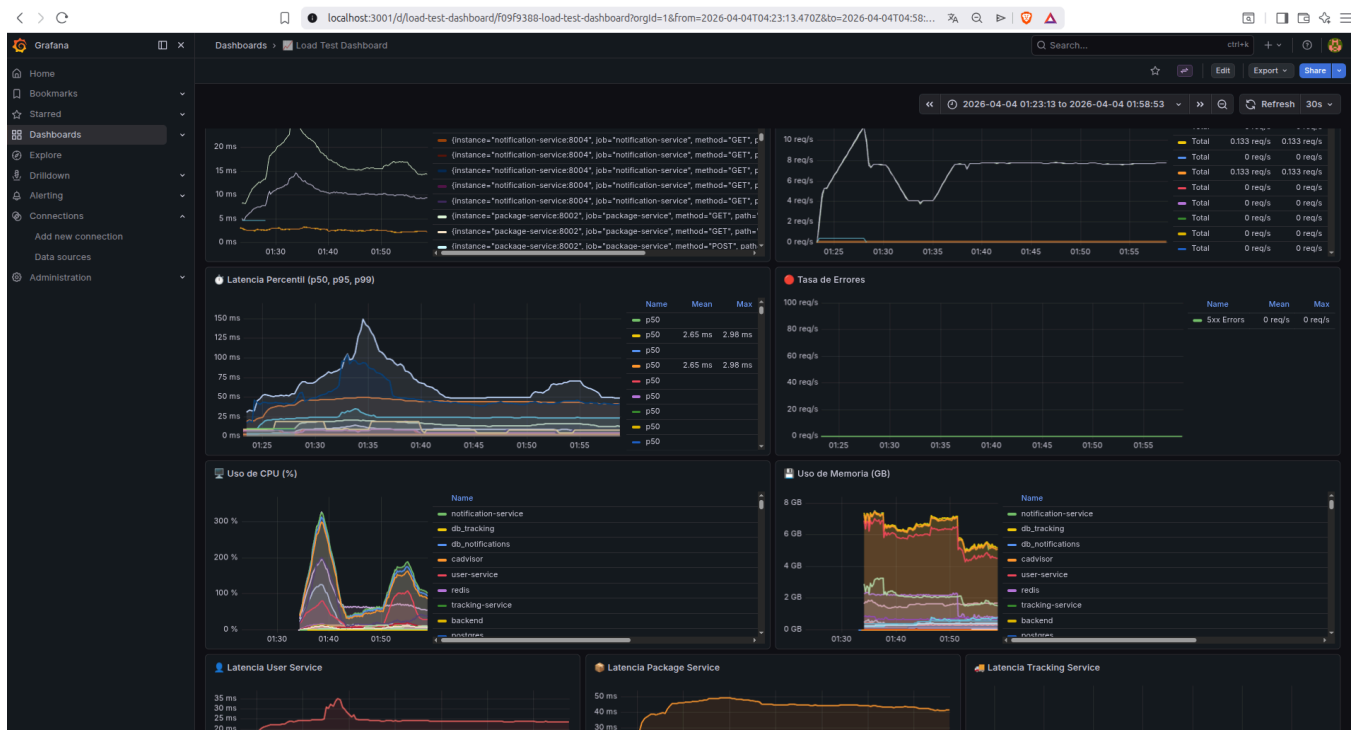


Figura 5: Dashboard de monitoreo (Microservicios)

7.4. Análisis de Métricas

A partir de la instrumentación con Prometheus y la visualización en Grafana, se evaluaron las métricas definidas en el punto 7.2: latencia, throughput, tasa de errores y uso de recursos. Los resultados obtenidos bajo inyección de carga muestran que el sistema monolítico mantiene un comportamiento funcional y estable en condiciones moderadas, aunque con limitaciones estructurales propias de su arquitectura.

La latencia observada (p95 = 45.1 ms) indica que la mayoría de las solicitudes se resuelven en tiempos bajos, lo cual es adecuado para el tipo de operaciones CRUD implementadas. Este valor sugiere que, en el escenario actual, la aplicación responde de manera ágil. No obstante, al concentrar toda la lógica en un único proceso, se espera que esta métrica aumente a medida que crezca la concurrencia o la complejidad de las operaciones.

El throughput registrado (alrededor de 12 req/s en la captura analizada) evidencia que el sistema soporta una carga sostenida sin degradación crítica. El desglose por endpoint muestra mayor actividad en rutas de creación y consulta de paquetes, coherente con el patrón de carga generado durante las pruebas. Esto confirma que el comportamiento observado en los gráficos se alinea con el tipo de tráfico inyectado.

La tasa de errores (1.49%) se mantuvo en un rango bajo, pero visible. Este comportamiento fue influenciado por errores controlados introducidos en pruebas de actualización de estado, con el propósito de validar la observabilidad del sistema. La métrica confirma que la instrumentación captura adecuadamente respuestas fallidas y permite monitorear su impacto en tiempo real.

Respecto al consumo de recursos, la CPU presentó picos breves cercanos a 0.24 cores sin saturación sostenida, y la memoria se mantuvo en un rango estable de aproximadamente 87 a 92 MB, sin señales de crecimiento anómalo. En conjunto, estos datos indican que el backend conserva margen operativo bajo la carga aplicada y no muestra síntomas inmediatos de inestabilidad por uso de recursos.

En términos arquitectónicos, el análisis confirma que el monolito cumple correctamente en el contexto evaluado, pero también expone su principal restricción: el escalado se realiza de forma global, no por dominio funcional. Esto limita la eficiencia ante crecimiento desigual de carga entre módulos. Por ello, la evidencia recolectada respalda una migración gradual hacia microservicios, priorizando la extracción de los componentes con mayor presión de tráfico (especialmente tracking), para habilitar escalado independiente y mayor resiliencia ante fallos.

7.5. Estimación de Costos

Concepto	Monolito	Microservicios
Infraestructura	Bajo costo inicial. Requiere pocos contenedores (frontend, backend, DB). Uso eficiente de CPU y memoria en un solo proceso.	Mayor costo inicial. Requiere múltiples contenedores (gateway, servicios, bases de datos, Redis, Prometheus, Grafana). Overhead por red interna y orquestación.
Escalabilidad	Limitada. El escalado es vertical o global (se escala todo el sistema aunque solo una parte lo necesite). Ineficiente en uso de recursos	Alta. Escalado horizontal por servicio. Permite invertir recursos solo en los componentes con mayor carga (ej: tracking). Optimiza costos a largo plazo.
Operación	Baja complejidad operativa. Despliegue, monitoreo y debugging centralizados. Menor costo de mantenimiento inicial	Mayor complejidad operativa. Requiere gestión de múltiples servicios, observabilidad distribuida y manejo de fallos. Mayor costo en operación y DevOps.

7.6. Análisis

La comparación de costos evidencia que el monolito presenta una ventaja clara en el corto plazo, debido a su menor complejidad y menor consumo de recursos. Su operación centralizada simplifica el despliegue y mantenimiento, pero limita la escalabilidad, ya que cualquier incremento de carga obliga a escalar todo el sistema, generando ineficiencia en el uso de recursos.

En contraste, la arquitectura de microservicios implica un mayor costo inicial, tanto en infraestructura como en operación, debido a la necesidad de múltiples servicios, herramientas de soporte y una gestión más compleja. Sin embargo, este costo se justifica por los beneficios que introduce.

La principal ventaja es la escalabilidad independiente, que permite asignar recursos de manera focalizada según la demanda de cada servicio, optimizando el uso de infraestructura en escenarios de crecimiento. Además, mejora la resiliencia del sistema, ya que los fallos se aíslan y no comprometen toda la aplicación.

8. Trade-offs

■ Beneficios:

- La comparación de costos evidencia que el monolito presenta una ventaja clara en el corto plazo, debido a su menor complejidad y menor consumo de recursos. Su operación centralizada simplifica el despliegue y mantenimiento, pero limita la escalabilidad, ya que cualquier incremento de carga obliga a escalar todo el sistema, generando ineficiencia en el uso de recursos.
- En contraste, la arquitectura de microservicios implica un mayor costo inicial, tanto en infraestructura como en operación, debido a la necesidad de múltiples servicios, herramientas de soporte y una gestión más compleja. Sin embargo, este costo se justifica por los beneficios que introduce.
- La principal ventaja es la escalabilidad independiente, que permite asignar recursos de manera focalizada según la demanda de cada servicio, optimizando el uso de infraestructura en escenarios de crecimiento. Además, mejora la resiliencia del sistema, ya que los fallos se aíslan y no comprometen toda la aplicación.
- En conclusión, el monolito es una solución eficiente para etapas iniciales o cargas bajas, mientras que los microservicios representan una inversión estratégica orientada a la escalabilidad, flexibilidad y robustez del sistema en el largo plazo.

■ Desventajas:

- Mayor complejidad operativa: Se requiere gestionar múltiples servicios, redes, bases de datos y herramientas de monitoreo.
- Mayor costo de infraestructura: Más contenedores, más consumo de recursos y necesidad de herramientas adicionales.
- Latencia de red: La comunicación entre servicios introduce overhead respecto a llamadas internas del monolito.
- Consistencia eventual: No se garantiza consistencia inmediata entre servicios, lo que complica la lógica de negocio.
- Mayor esfuerzo de desarrollo: Implementar patrones como Saga o Circuit Breaker aumenta la complejidad del sistema.

■ Cuando NO usar microservicios:

- Sistemas pequeños o de baja complejidad.
- Proyectos en etapas iniciales sin validación de negocio.
- Equipos de desarrollo reducidos sin experiencia en sistemas distribuidos.
- Aplicaciones con baja demanda o sin necesidad de escalabilidad.
- Cuando la simplicidad y rapidez de desarrollo son más críticas que la escalabilidad.

9. Conclusiones

El proceso de análisis y migración desde una arquitectura monolítica hacia microservicios permitió identificar tanto las limitaciones del sistema actual como las oportunidades de mejora en términos de escalabilidad, resiliencia y organización del dominio.

El sistema monolítico demostró ser funcional y eficiente en escenarios de carga moderada, con un bajo consumo de recursos y una operación sencilla. Sin embargo, su alto acoplamiento, la dificultad para escalar de manera selectiva y la concentración de responsabilidades en un único bloque evidencian limitaciones estructurales que pueden impactar negativamente a medida que el sistema crece.

La propuesta basada en microservicios introduce una arquitectura más robusta y alineada con escenarios de mayor demanda. La separación por dominios, el uso de patrones arquitectónicos como API Gateway, Saga, Circuit Breaker y Strangler Pattern, junto con la adopción de una estrategia de base de datos por servicio, permiten construir un sistema más flexible, escalable y resiliente.

No obstante, esta mejora no es gratuita. La migración implica un aumento en la complejidad técnica, operativa y de infraestructura, lo que exige una mayor madurez en el desarrollo y en la gestión del sistema. Por ello, la adopción de microservicios debe entenderse como una decisión estratégica, orientada a resolver problemas reales de escalabilidad y evolución, y no como una solución universal.

En conclusión, la migración propuesta representa una evolución necesaria para soportar el crecimiento del sistema de trazabilidad de paquetes, permitiendo una mejor distribución de la carga, mayor tolerancia a fallos y una base sólida para futuras extensiones. El enfoque incremental mediante Strangler Pattern reduce los riesgos asociados, asegurando una transición controlada y validable en cada etapa del proceso.

10. Referencias

[Atlassian](#)

Atlassian. (s.f.). *Comparación entre la arquitectura monolítica y la arquitectura de microservicios*.

[Amazon Web Services \(AWS\)](#)

Amazon Web Services. (s.f.). *Diferencia entre arquitectura monolítica y microservicios*.

[CodeSee](#)

CodeSee. (s.f.). *Microservices vs monolith*.

[Ackstorm](#)

Ackstorm. (s.f.). *Pros y contras de usar microservicios*.

[Grafana Labs](#)

Grafana Labs. (s.f.). *Grafana documentation*.